

RENDERING THE VIRTUAL DOM

Created: 2025-11-27 Thu 14:13

WHAT WE WILL COVER

WHAT WE WILL COVER

- Defining the virtual DOM

WHAT WE WILL COVER

- Defining the virtual DOM
- Understanding the problems the virtual DOM solves

WHAT WE WILL COVER

- Defining the virtual DOM
- Understanding the problems the virtual DOM solves
- Implementing functions to create virtual DOM nodes

WHAT WE WILL COVER

- Defining the virtual DOM
- Understanding the problems the virtual DOM solves
- Implementing functions to create virtual DOM nodes
- Defining the concept of a stateless component

PROBLEMS WITH PREVIOUS APPROACH?

PROBLEMS WITH PREVIOUS APPROACH?

- use of the Document API is the most burdensome to write.

PROBLEMS WITH PREVIOUS APPROACH?

- use of the Document API is the most burdensome to write.
- mixing application logic and DOM manipulation is problematic

PROBLEMS WITH PREVIOUS APPROACH?

- use of the Document API is the most burdensome to write.
- mixing application logic and DOM manipulation is problematic
- overly verbose
 - events that change the state must handle updates to the DOM.

PROBLEMS WITH PREVIOUS APPROACH?

- use of the Document API is the most burdensome to write.
- mixing application logic and DOM manipulation is problematic
- overly verbose
 - events that change the state must handle updates to the DOM.
- **imperative code** - verbose steps of *how to do*

POSSIBLE IMPROVEMENTS:

POSSIBLE IMPROVEMENTS:

- abstract the manipulation of the DOM

POSSIBLE IMPROVEMENTS:

- abstract the manipulation of the DOM
- focus on the application logic

POSSIBLE IMPROVEMENTS:

- abstract the manipulation of the DOM
- focus on the application logic
- **declarative code** - focus on *what to do*

POSSIBLE IMPROVEMENTS:

- abstract the manipulation of the DOM
- focus on the application logic
- **declarative code** - focus on *what to do*
- create *high-level* and *low-level* code
 - *high-level* - application logics
 - *low-level* - DOM manipulation

SEPARATING CONCERNS

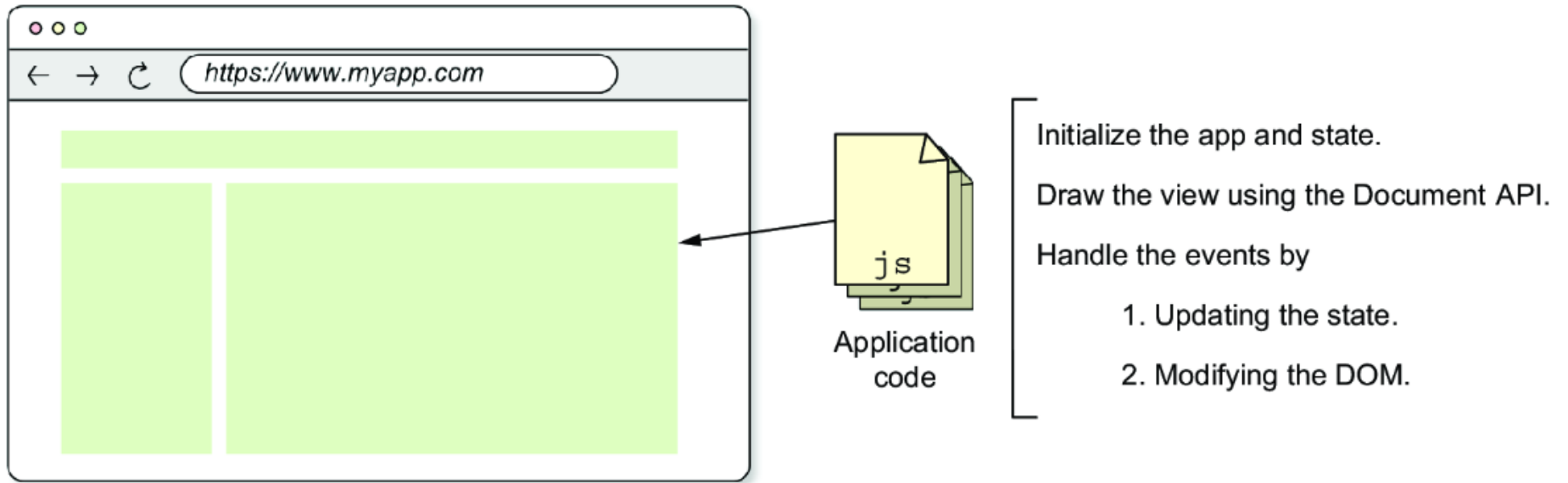


Figure 1: State of the application code

PRIMARY GOALS

- Goal: Separate the application code from the DOM manipulation code

Dfn: Separation of concerns means splitting the code so that the parts that carry out different responsibilities are separated, which helps the developer understand and maintain the code.

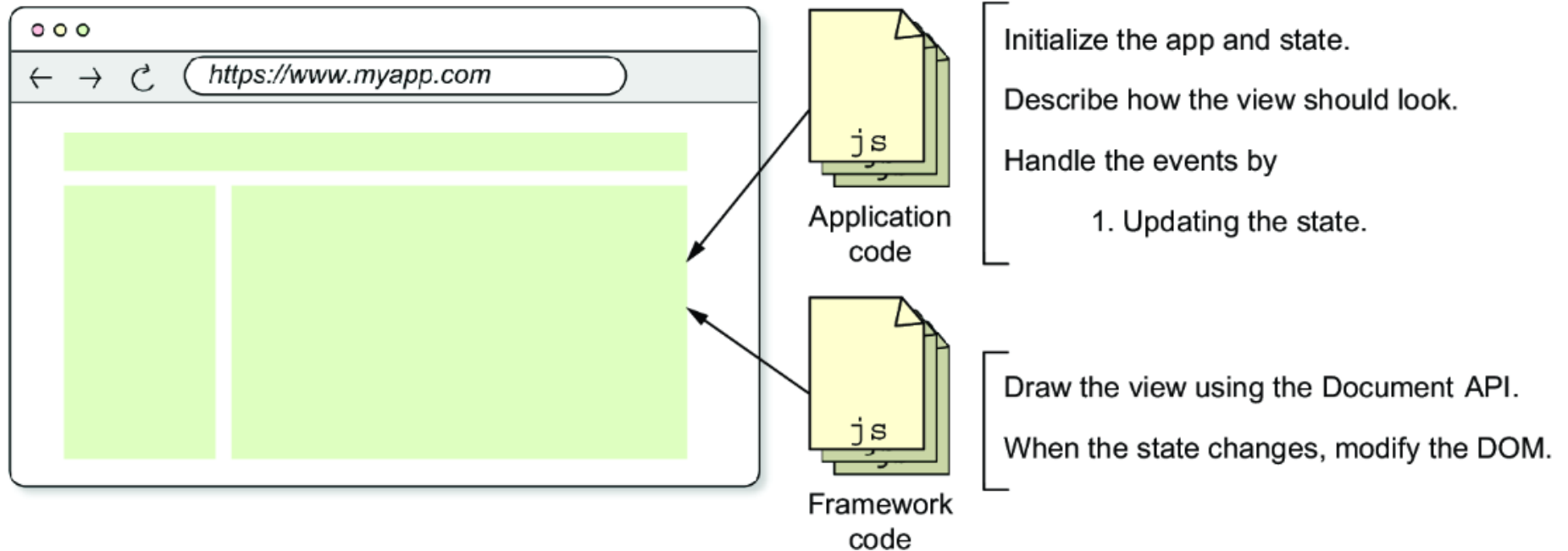


Figure 2: Desired end state of the application code

IMPROVEMENTS:

Developer productivity

The application developer doesn't need to write DOM-manipulation code; instead, they can focus on the application logic.

Code maintainability

The DOM manipulation and application logic aren't mixed, which makes the code more succinct and easier to understand.

Framework performance

The framework author is likely to understand how to produce efficient and optimized DOM-manipulation code better than the application

THE VIRTUAL DOM

*Dfn: The virtual DOM is a lightweight, in-memory representation of the actual (browser's) **DOM**.*

THE VIRTUAL DOM

*Dfn: The virtual DOM is a lightweight, in-memory representation of the actual (browser's) **DOM**.*

It allows describing the view in a **declarative** way!

Some notes:

Some notes:

- **DOM** - in-memory tree structure managed by the browser engine, representing the HTML structure of the web page.

Some notes:

- **DOM** - in-memory tree structure managed by the browser engine, representing the HTML structure of the web page.
- **Virtual DOM (vdom)** - JavaScript-based in-memory tree of virtual nodes that mirrors the structure of the actual DOM.

Some notes:

- **DOM** - in-memory tree structure managed by the browser engine, representing the HTML structure of the web page.
- **Virtual DOM (vdom)** - JavaScript-based in-memory tree of virtual nodes that mirrors the structure of the actual DOM.
- Each node in *vdom* is called a **virtual node (vnode)**.

- DOM nodes are heavy objects with hundreds of properties.
- *vnodes* are cheap and lightweight (basically JavaScript objects)

A virtual DOM representation of this HTML needs to contain the same information as the DOM:

A virtual DOM representation of this HTML needs to contain the same information as the DOM:

- What nodes are in the tree and their attributes

A virtual DOM representation of this HTML needs to contain the same information as the DOM:

- What nodes are in the tree and their attributes
- The hierarchy of the nodes in the tree

A virtual DOM representation of this HTML needs to contain the same information as the DOM:

- What nodes are in the tree and their attributes
- The hierarchy of the nodes in the tree
- The relative positions of the nodes in the tree

For the following form

```
<form action="/login" class="login-form">  
  <input type="text" name="user" />  
  <input type="password" name="pass" />  
  <button>Log in</button>  
</form>
```


We can conclude:

We can conclude:

- The `form` element is the root node:

We can conclude:

- The `form` element is the root node:
 - attributes: `action`, `class`

We can conclude:

- The `form` element is the root node:
 - attributes: `action`, `class`
 - implicit `click` event handler

We can conclude:

- The `form` element is the root node:
 - attributes: `action`, `class`
 - implicit `click` event handler
- `input` elements have `name` and `type`

We can conclude:

- The `form` element is the root node:
 - attributes: `action`, `class`
 - implicit `click` event handler
- `input` elements have `name` and `type`
- position of the elements is important

Proposed *vdom* structure

```
{
  type: 'element',
  tag: 'form',
  props: { action: '/login', class: 'login-form' },
  children: [
    {
      type: 'element',
      tag: 'input',
      props: { type: 'text', name: 'user' }
    },
    {
      type: 'element',
      tag: 'input',
      props: { type: 'password', name: 'pass' }
    },
  ],
}
```

```
{
  type: 'element',
  tag: 'button',
  props: { on: { click: () => login() } },
  children: [
    {
      type: 'text',
      value: 'Log in'
    }
  ]
}
```


NODE TYPES

Each node in the virtual DOM is an object with a *nodeType* property.

NODE TYPES

Each node in the virtual DOM is an object with a *nodeType* property.

- `element` - Represents a regular **HTML element**, such as `<form>`, `<input>`, or `<button>`

NODE TYPES

Each node in the virtual DOM is an object with a *nodeType* property.

- `element` - Represents a regular **HTML element**, such as `<form>`, `<input>`, or `<button>`
- `text` - Represents a **text** node, such as the ' Log in ' text of the `<button>` element in the example

NODE TYPES

Each node in the virtual DOM is an object with a *nodeType* property.

- `element` - Represents a regular **HTML element**, such as `<form>`, `<input>`, or `<button>`
- `text` - Represents a **text** node, such as the ' Log in ' text of the `<button>` element in the example
- `fragment` - Represents a node that don't have a parent node until they are attached to the DOM.

Each type of node has a set of properties to describe it:

Each type of node has a set of properties to describe it:

- **text nodes** - `type` and `value` (string of text)

Each type of node has a set of properties to describe it:

- **text nodes** - `type` and `value` (string of text)
- **element virtual nodes**
 - `tag` - The tag name of the HTML element.
 - `props` - The attributes of the HTML element including event handlers.
 - `children` - ordered children of node.

VDOM REPRESENTATION

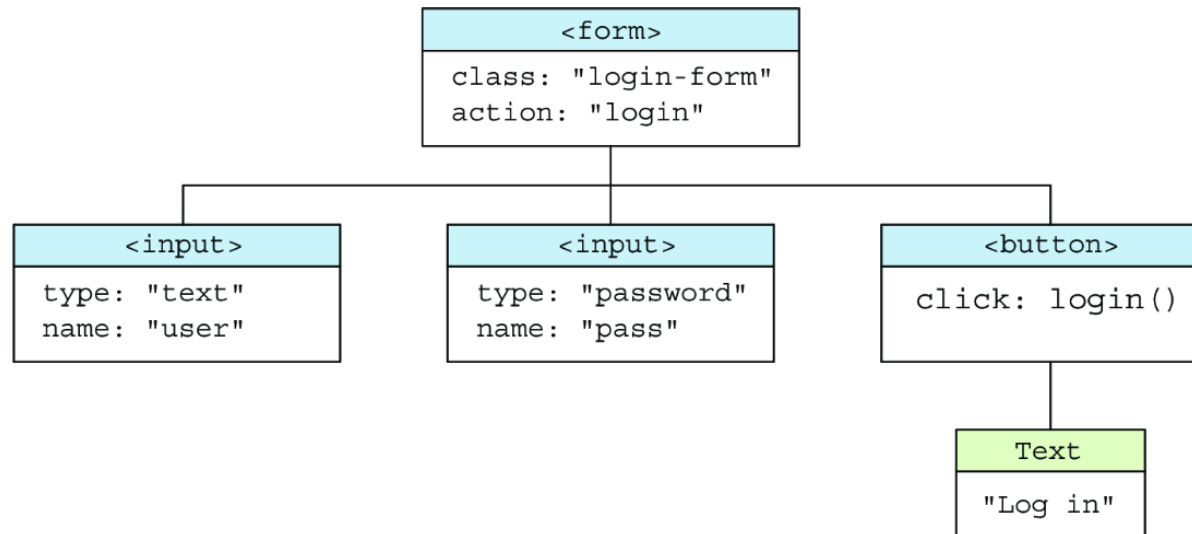


Figure 3: VDOM representation of the previous example

INITIAL SETUP

```
├── runtime
│   └── src
│       ├── h.js
│       ├── index.js
│       └── utils
│           └── array.js
```

NODE TYPES

Inside `h.js`:

```
export const DOM_TYPES = {  
  TEXT: 'text',  
  ELEMENT: 'element',  
  FRAGMENT: 'fragment',  
}
```

ELEMENT NODES

Create a function that creates virtual node object.

- *React* - `React.createElement()`
- *Vue.js* - `h()`
- *Mithril* - `m()`

```
import { withoutNulls } from './utils/arrays'

export function h(tag, props = {}, children = []) {
  return {
    tag,
    props,
    children: mapTextNodes(withoutNulls(children)),
    type: DOM_TYPES.ELEMENT,
  }
}
```

CONDITIONAL RENDERING

Remove possible null values from the arrays.

Remove the Add button

CONDITIONAL RENDERING

Remove possible null values from the arrays.

Remove the Add button

```
{
  tag: 'div',
  children: [
    { tag: 'input', props: { type: 'text' } },
    addTodoInput.value.length > 2
      ? { tag: 'button', children: ['Add'] }
      : null
  ]
}
```

Adding `null` when
`addTodoInput.value.length > 2` is false

```
{
  tag: 'div',
  children: [
    { tag: 'input', props: { type: 'text' } },
    null
  ]
}
```

withoutNulls implementation in utils/array.js

```
export function withoutNulls(arr) {  
  return arr.filter((item) => item != null)  
}
```


MAPPING STRINGS TO TEXT NODES

MAPPING STRINGS TO TEXT NODES

Convert text strings into text virtual nodes.

MAPPING STRINGS TO TEXT NODES

Convert text strings into text virtual nodes.

```
// Instead of:  
h('div', {}, [hString('Hello '), hString('world!')])  
  
// write:  
h('div', {}, ['Hello ', 'world!'])
```

Implementation of mapTextNodes function

```
function mapTextNodes(children) {  
  return children.map((child) =>  
    typeof child === 'string' ? hString(child) : child  
  )  
}
```

CREATING TEXT NODES

Implementation of hString function

```
export function hString(str) {  
  return { type: DOM_TYPES.TEXT, value: str }  
}
```

CREATING FRAGMENT NODES

CREATING FRAGMENT NODES

Fragment will group multiple nodes not currently attached to the DOM. Container for an array of virtual nodes.

Implementation of hFragment function

```
export function hFragment(vNodes) {  
  return {  
    type: DOM_TYPES.FRAGMENT,  
    children: mapTextNodes(withoutNulls(vNodes)),  
  }  
}
```


TESTING THE VDOM FUNCTIONS

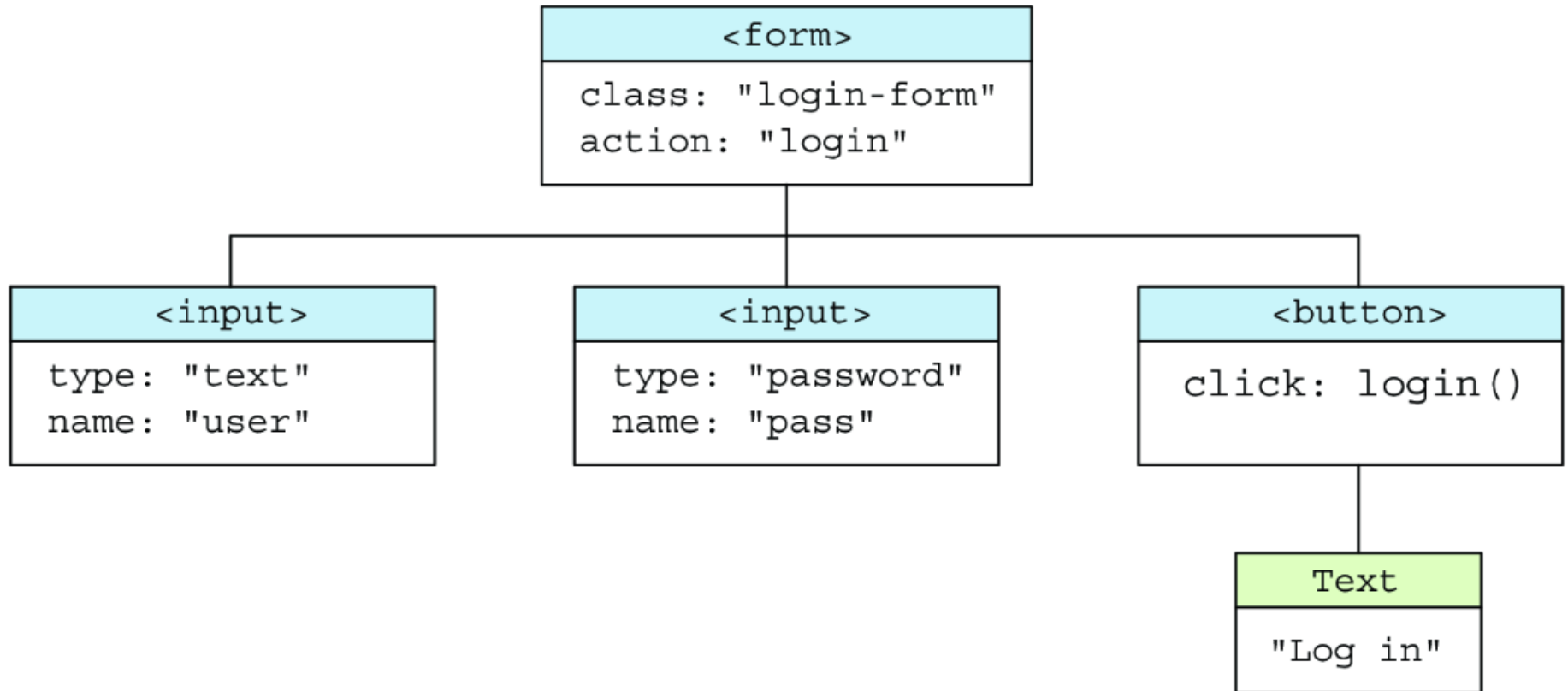


Figure 4: Example VDOM tree

```
<form class="login-form" action="login">  
  <input type="text" name="user">  
  <input type="password" name="pass">  
  <button>Log in</button>  
</form>
```

```
h('form', { class: 'login-form', action: 'login' }, [  
  h('input', { type: 'text', name: 'user' }),  
  h('input', { type: 'password', name: 'pass' }),  
  h('button', { on: { click: login } }, ['Log in'])  
])
```

COMPONENTS

COMPONENTS

- Mini application (web page) in itself

COMPONENTS

- Mini application (web page) in itself
- Own lifecycle

COMPONENTS

- Mini application (web page) in itself
- Own lifecycle
- In charge of its own rendering

COMPONENTS

- Mini application (web page) in itself
- Own lifecycle
- In charge of its own rendering
- Communicating with the rest of the application

COMPONENTS

- Mini application (web page) in itself
- Own lifecycle
- In charge of its own rendering
- Communicating with the rest of the application
 - emitting events

COMPONENTS

- Mini application (web page) in itself
- Own lifecycle
- In charge of its own rendering
- Communicating with the rest of the application
 - emitting events
 - receiving props

What we'll do:

What we'll do:

We'll implement a limited and simple variation of a component. Components will be **pure functions**

What we'll do:

We'll implement a limited and simple variation of a component. Components will be **pure functions**

- that returns the VDOM

What we'll do:

We'll implement a limited and simple variation of a component. Components will be **pure functions**

- that returns the VDOM
- view of the component

What we'll do:

We'll implement a limited and simple variation of a component. Components will be **pure functions**

- that returns the VDOM
- view of the component
- no state (for now)

VDOM AS FUNCTION OF STATE

VDOM AS FUNCTION OF STATE

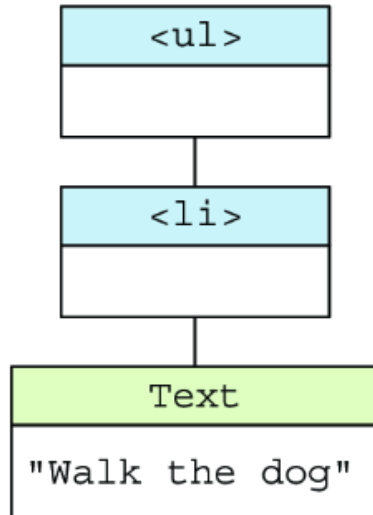
The view of the application is dependant of the state.
Every change of the state should force reevaluation of
the VDOM Every reevaluation of the VDOM may
produce update of the DOM

OUR TODO APPLICATION

A new to-do is added to the array of to-dos: the state changes.

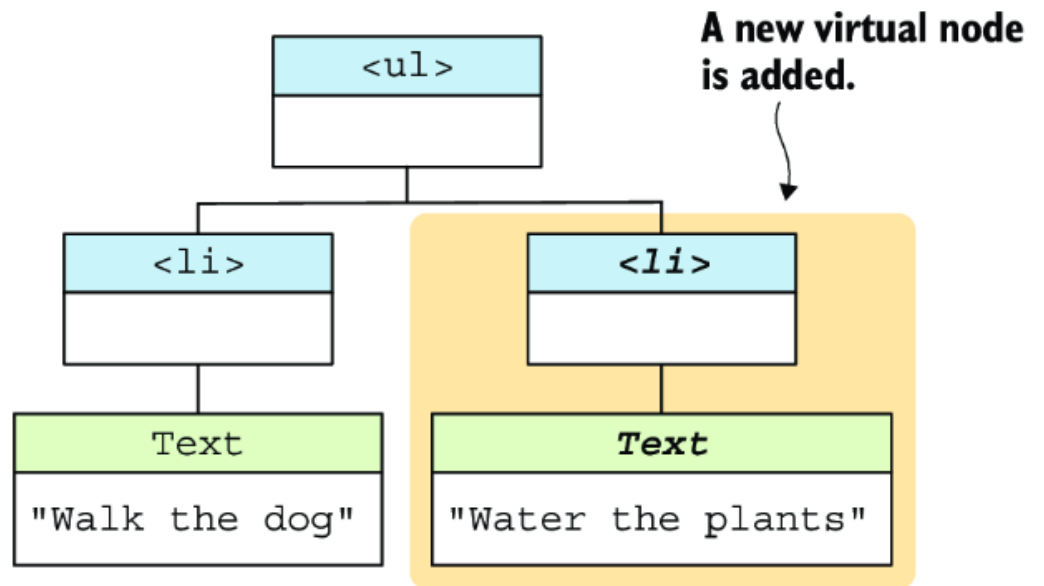
```
state = ["Walk the dog"]
```

Virtual DOM



```
state = ["Walk the dog", "Water the plants"]
```

Virtual DOM



HTML

```
<ul>
  <li>Walk the dog</li>
</ul>
```

HTML

```
<ul>
  <li>Walk the dog</li>
  <li>Water the plants</li>
</ul>
```

A new HTML element is added.

Figure 5: The view of an application is a function of the state.

```
function TodosList(todos) {  
  return h('ul', {}, todos.map((todo) => h('li', {}, [todo])))  
}
```

```
TodosList(['Walk the dog', 'Water the plants'])
```

```
{  
  tag: 'ul',  
  type: 'element',  
  children: [  
    { tag: 'li', children: [{ type: 'text', value: 'Walk the dog' } ] },  
    { tag: 'li', children: [{ type: 'text', value: 'Water the plant' } ] },  
  ]  
}
```

The result of the rendering:

```
<ul>  
  <li>Walk the dog</li>  
  <li>Water the plants</li>  
</ul>
```

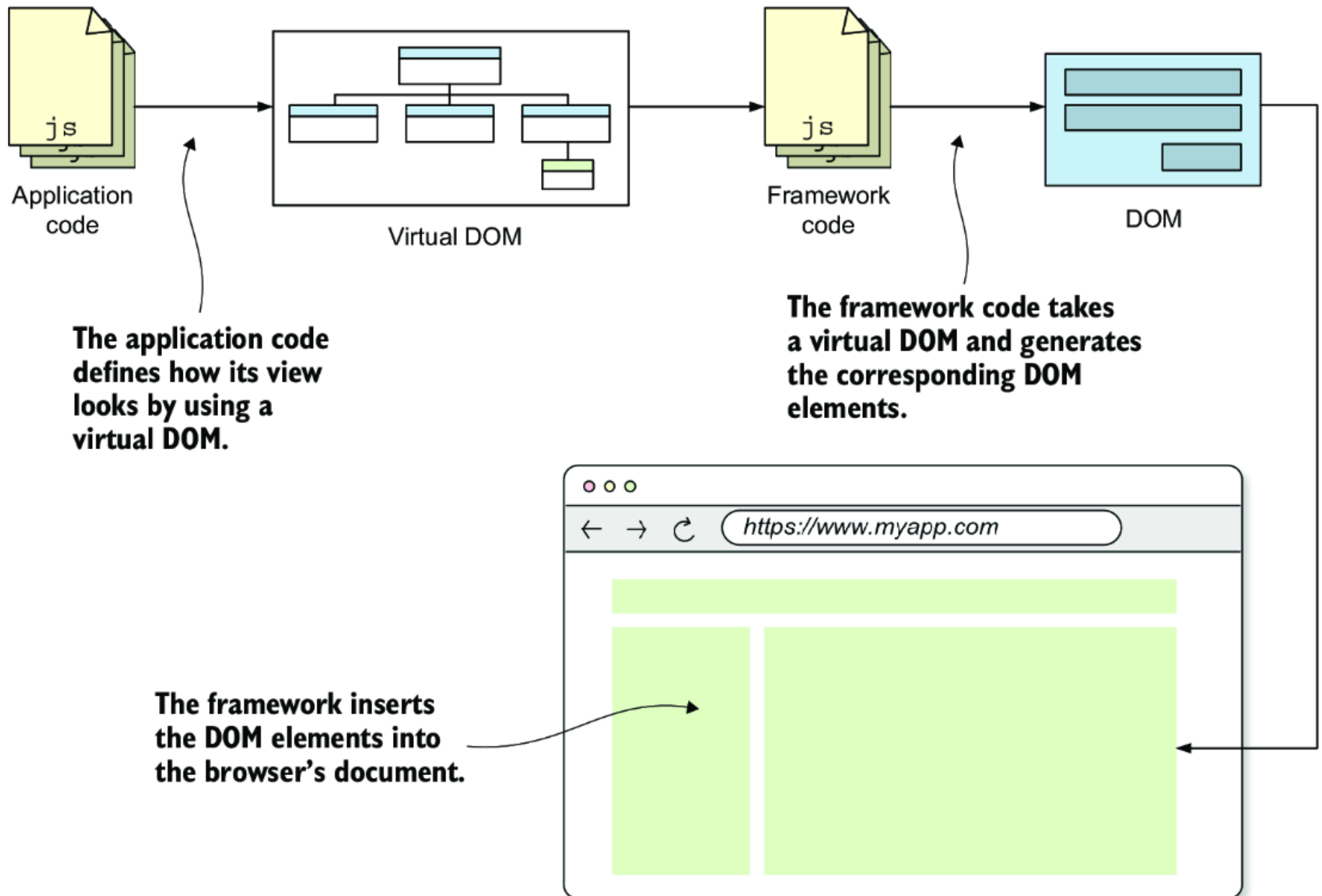


Figure 6: The application creates the virtual DOM that
the framework renders into the DOM

One nice benefit of using functions to produce the virtual DOM, receiving the application's state as an argument, is that we can break the view into smaller parts. We can easily compose *pure functions*—functions that don't produce any side effects and always return the same result for the same input arguments—to build more complex views.

BENEFITS OF *PURE FUNCTIONS*

BENEFITS OF *PURE FUNCTIONS*

- Pure functions can be composed nicely to build more complex functions.

BENEFITS OF *PURE FUNCTIONS*

- Pure functions can be composed nicely to build more complex functions.
- Break the view into smaller parts:
 - *components*
 - The arguments passed to a component are known as *props*,

BENEFITS OF *PURE FUNCTIONS*

- Pure functions can be composed nicely to build more complex functions.
- Break the view into smaller parts:
 - *components*
 - The arguments passed to a component are known as *props*,
- Components don't have state

BENEFITS OF *PURE FUNCTIONS*

- Pure functions can be composed nicely to build more complex functions.
- Break the view into smaller parts:
 - *components*
 - The arguments passed to a component are known as *props*,
- Components don't have state
yet!

My TODOs

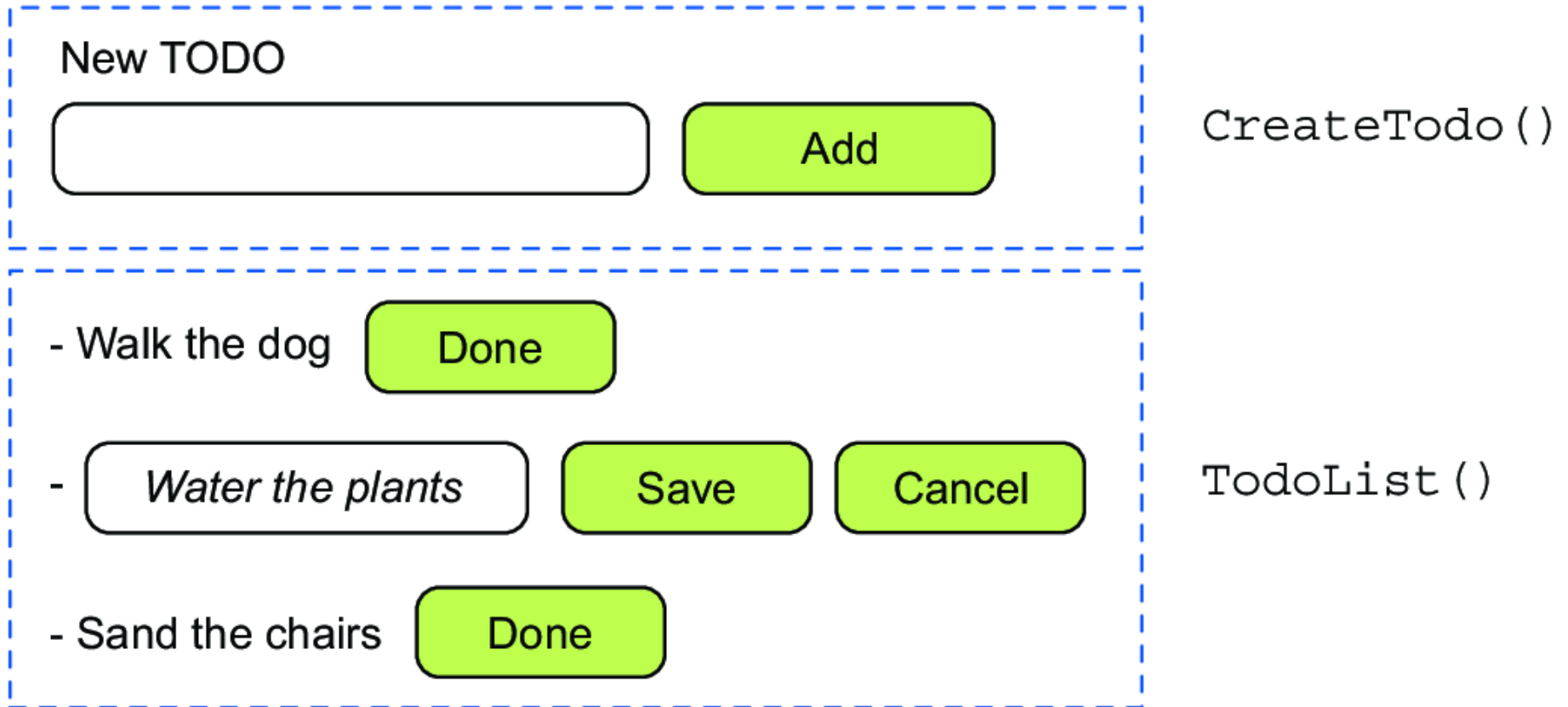


Figure 7: Splitting the TODO app into components

- Example component creation:

```
function App(state) {  
  return hFragment([  
    h('h1', {}, ['My TODOs']),  
    CreateTodo(state),  
    TodoList(state)  
  ])  
}
```

- Notice:
 - no parent node in *vdom* contains the header and the subcomponents
 - *vdom* creation functions are in PascalCase
 - The `TodoList()` component is broken into `TodoItem()`

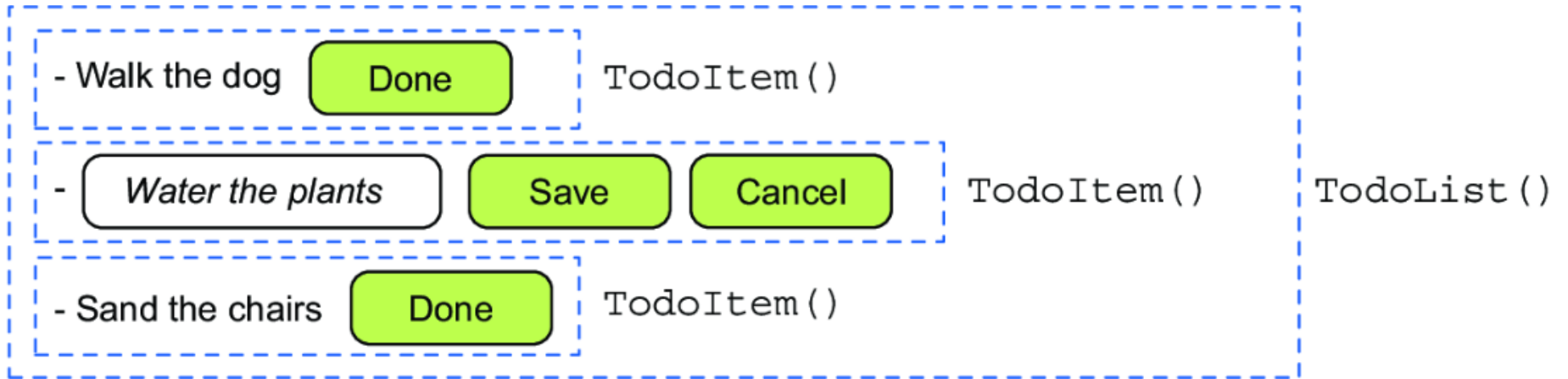


Figure 8: `TodoList` component

```
function TodoList(state) {  
  return h('ul', {},  
    children: state.todos.map(  
      (todo, i) => TodoItem(todo, i, state.editingIdxs)  
    )  
  )  
}
```

- Rendering different things depending on the read/edit mode
- Two different subcomponents
 - `TodoInReadMode()`
 - `TodoInEditMode()`

```
// idxInList is the index of this todo item in the list of todos.  
// editingIdxs is a Set of indexes of todos that are being edited.  
function TodoItem(todo, idxInList, editingIdxs) {  
  const isEditing = editingIdxs.has(idxInList)  
  
  return h('li', {}, [  
    isEditing  
      ? TodoInEditMode(todo, idxInList)  
      : TodoInReadMode(todo, idxInList)  
  ]  
)  
}
```

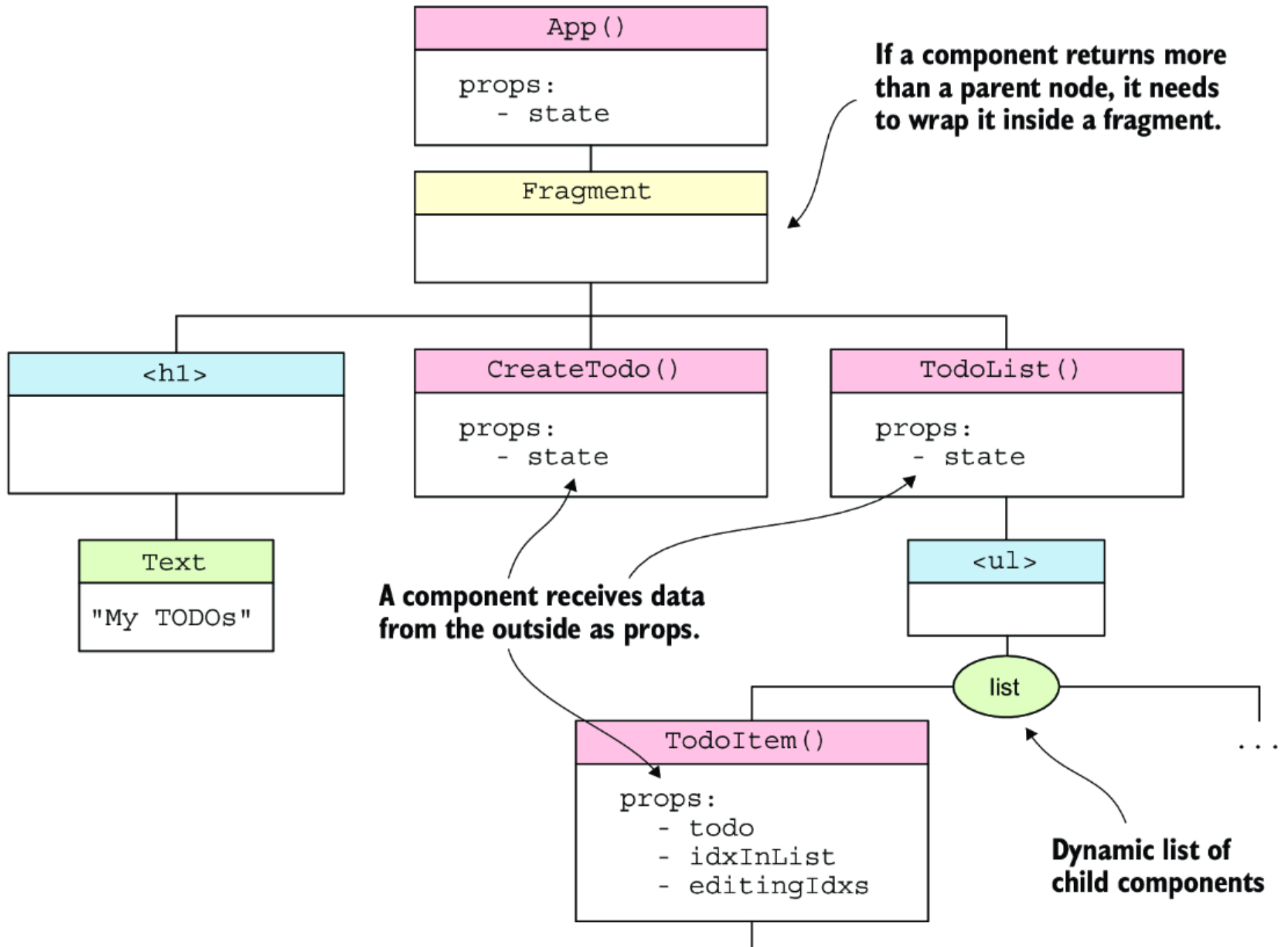


Figure 9: Hierarchy of components for the TODO

- Tree of components
 - `App ( )` - root of the tree
 - `<h1>`
 - `CreateTodo ( )`
 - `TodoList ( )`

- Tree of components
 - `App ( )` - root of the tree
 - `<h1>`
 - `CreateTodo ( )`
 - `TodoList ( )`
- A component returns a single virtual DOM node

- Tree of components
 - `App ( )` - root of the tree
 - `<h1>`
 - `CreateTodo ( )`
 - `TodoList ( )`
- A component returns a single virtual DOM node
- The 3 children are grouped in a fragment node.

- `TodoList()` - has a single child `<ul>`
- The `<ul>` can have multiple children of type `TodoItem()`
- One `TodoItem()` has two children:
 - `TodoInEditMode()`
 - `TodoInReadMode()`

NEXT

- We'll discuss mounting and destroying the Virtual DOM